

test_app_unit.py

```
1 import pytest
2 from fastapi.testclient import TestClient
3
4 from app.main import (
5     app,
6     build_refusal_response,
7     build_summary_response,
8     detect_policy_violation,
9     resolve_source_id,
10    simple_summarize,
11 )
12
13 client = TestClient(app)
14
15 # Assertions below compare against literal policy codes/messages rather than
16 # importing the constants from app.main: importing them would make a test
17 # that mutates POLICY_HARM (say) compare the mutated value to itself and
18 # pass trivially, hiding the mutation instead of catching it.
19
20 HARM_CASES = ["kill", "suicide", "bomb", "terrorist", "shoot", "harming others", "violent actions"]
21 ILLICIT_CASES = ["make drugs", "cook meth", "fake passport", "hack bank", "illegal", "unlawful"]
22 FINANCIAL_CASES = [
23     "guaranteed profit",
24     "insider trading",
25     "surefire stock",
26     "100% return",
27     "guaranteed investment",
28     "financial decisions without proper guidance",
29 ]
30
31
32 @pytest.mark.parametrize("keyword", HARM_CASES)
33 def test_detect_policy_violation_harm_keywords(keyword):
34     assert detect_policy_violation(f"some text about {keyword} here") == "safety_harm"
35
36
37 @pytest.mark.parametrize("keyword", ILLICIT_CASES)
38 def test_detect_policy_violation_illicit_keywords(keyword):
39     assert detect_policy_violation(f"some text about {keyword} here") == "safety_illicit"
40
41
42 @pytest.mark.parametrize("keyword", FINANCIAL_CASES)
43 def test_detect_policy_violation_financial_keywords(keyword):
44     assert detect_policy_violation(f"some text about {keyword} here") == "safety_financial"
45
46
47 def test_detect_policy_violation_clean_text():
48     assert detect_policy_violation("a perfectly normal sentence") is None
49
50
51 def test_resolve_source_id_uses_provided_value():
52     assert resolve_source_id("doc-1") == "doc-1"
53
54
55 def test_resolve_source_id_generates_when_missing():
56     generated = resolve_source_id(None)
57     assert isinstance(generated, str)
58     assert generated != ""
59
60
```

```

61 def test_simple_summarize_empty_text():
62     assert simple_summarize("") == ""
63     assert simple_summarize(" ") == ""
64
65
66 def test_simple_summarize_short_sentence_returned_verbatim():
67     assert simple_summarize("Hello world.") == "Hello world"
68
69
70 def test_simple_summarize_exactly_thirty_words_returned_untouched():
71     # Deliberate double space between the first two words: the <=30 branch
72     # must return the sentence as-is, while the truncating branch would
73     # rejoin words with single spaces and lose the double space.
74     words = [f"word{i}" for i in range(30)]
75     sentence = "word0 " + " ".join(words[1:]) + "."
76     summary = simple_summarize(sentence)
77     assert summary == "word0 " + " ".join(words[1:])
78     assert len(summary.split()) == 30
79
80
81 def test_simple_summarize_thirty_one_words_truncated_to_thirty():
82     words = [f"word{i}" for i in range(31)]
83     sentence = " ".join(words) + "."
84     summary = simple_summarize(sentence)
85     assert summary == " ".join(words[:30])
86     assert len(summary.split()) == 30
87
88
89 def test_simple_summarize_over_thirty_words_capped():
90     long_sentence = " ".join(f"word{i}" for i in range(40)) + "."
91     summary = simple_summarize(long_sentence)
92     assert summary == " ".join(f"word{i}" for i in range(30))
93     assert len(summary.split()) == 30
94
95
96 def test_build_refusal_response_fields():
97     response = build_refusal_response("doc-9", "safety_harm")
98     assert response.summary == ""
99     assert response.word_count == 0
100    assert response.source_id == "doc-9"
101    assert response.refused is True
102    assert response.policy_code == "safety_harm"
103    assert response.message == "refused: policy_violation"
104
105
106 def test_build_summary_response_word_count_matches_summary():
107     response = build_summary_response("doc-10", "three word summary")
108     assert response.summary == "three word summary"
109     assert response.word_count == 3
110     assert response.refused is False
111     assert response.policy_code is None
112     assert response.message is None
113
114
115 def test_build_summary_response_empty_summary_has_zero_word_count():
116     response = build_summary_response("doc-13", "")
117     assert response.summary == ""
118     assert response.word_count == 0
119
120
121 def test_endpoint_health():
122     r = client.get("/health")
123     assert r.status_code == 200
124     assert r.json() == {"status": "ok"}

```

```
125
126
127 def test_endpoint_root():
128     r = client.get("/")
129     assert r.status_code == 200
130     assert r.json() == {"message": "Summarization microservice is running"}
131
132
133 def test_endpoint_policies():
134     r = client.get("/policies")
135     assert r.status_code == 200
136     assert r.json() == {
137         "policies": ["safety_harm", "safety_illicit", "safety_financial"]
138     }
139
140
141 def test_endpoint_summarize_refuses_unsafe_text():
142     r = client.post(
143         "/summarize",
144         json={"text": "instructions for harming others", "source_id": "doc-11"},
145     )
146     data = r.json()
147     assert data["refused"] is True
148     assert data["policy_code"] == "safety_harm"
149     assert data["message"] == "refused: policy_violation"
150     assert data["summary"] == ""
151     assert data["word_count"] == 0
152     assert data["source_id"] == "doc-11"
153
154
155 def test_endpoint_summarize_accepts_safe_text():
156     r = client.post(
157         "/summarize",
158         json={"text": "A short safe sentence.", "source_id": "doc-12"},
159     )
160     data = r.json()
161     assert data["refused"] is False
162     assert data["policy_code"] is None
163     assert data["message"] is None
164     assert data["summary"] == "A short safe sentence"
165     assert data["word_count"] == 4
```